RoboFleet – Checkpoint 2: notatki do slajdów

Łączny czas: 7 min 55 s (limit 10 min).

Podział: Bartek slajdy 1–3, Mikołaj 4–7, Anna 8–10, Patrick 11–12.

Slajd 1 – Strona tytułowa (Bartek, 15 s)

Dzień dobry, zespół RoboFleet. Na Checkpoint 2 przygotowaliśmy diagramy stanów pięciu kluczowych obiektów biznesowych i diagramy aktywności pięciu kluczowych funkcjonalności. Dziś pokażemy sześć najważniejszych. Ja omówię zakres i przebieg symulacji, Mikołaj pozostałe obiekty, Anna krok symulacji, a Patrick start przebiegu i wnioski.

Slajd 2 – Zakres (Bartek, 35 s)

Wybraliśmy obiekty, których stany są stanami procesu biznesowego: przebieg symulacji, zlecenie transportowe, robot w trakcie przebiegu, magazyn i konto użytkownika. Każdy diagram ma przejścia standardowe, nietypowe i stany specjalne. Diagramy aktywności opisują funkcje wykonywane powtarzalnie: krok symulacji, start przebiegu, dodanie zlecenia, zapis magazynu i podgląd na żywo. W dokumencie każdy diagram ma tabelę przejść nietypowych albo ścieżek alternatywnych. Dziś omówimy cztery diagramy stanów i dwa diagramy aktywności; pozostałe są w PDF-ie.

Slajd 3 – SM-1 Przebieg symulacji (Bartek, 60 s)

Przebieg symulacji. Powstaje jako szkic DRAFT, w którym klient dodaje zlecenia i ustawia parametry. Polecenie start waliduje przebieg na zamrożonej migawce magazynu; jeśli są błędy, przebieg zostaje szkicem i dostaje raport. Stan aktywny jest stanem złożonym: RUNNING i PAUSED, z pauzą i wznowieniem. Ten stan istnieje tylko w pamięci RunManagera, w bazie zapisujemy sam status. Z prawej cztery stany końcowe, tylko do odczytu, każdy z zapisaną przyczyną. Ścieżki nietypowe: FAILED po limicie ticków, po K tickach bez postępu, czyli zakleszczeniu, albo po błędzie silnika wykrytym przez SafetyMonitor; STOPPED po stop użytkownika, dozwolone także z pauzy, otwarte zlecenia są anulowane; INTERRUPTED po restarcie serwera, oznaczane przy starcie aplikacji dla przebiegów zapisanych jako RUNNING lub PAUSED. Teraz Mikołaj o zleceniu.

Slajd 4 – SM-2 Zlecenie transportowe (Mikołaj, 50 s)

Zlecenie transportowe. Powstaje w stanie PENDING, który ma dwa podstany wyliczane z releaseTick: czeka na zwolnienie, potem jest w kolejce FIFO. Po przydziale robota zlecenie jest ASSIGNED, po załadunku PICKED_UP, po rozładunku DELIVERED. Ścieżka nietypowa: gdy robot N razy z rzędu nie znajdzie trasy do odbioru, wywołujemy unassign i zlecenie wraca do kolejki; to nowa operacja wynikająca z analizy stanów. Stan złożony Otwarte pozwala jednym przejściem opisać skutki końca przebiegu: FAILED przy błędzie, CANCELLED po stop użytkownika. Zlecenie w transporcie nie wraca do kolejki: robot z towarem czeka na trasę.

Slajd 5 – SM-3 Robot w trakcie przebiegu (Mikołaj, 40 s)

Robot w trakcie przebiegu istnieje tylko w pamięci silnika. Cykl: IDLE, dojazd do odbioru, załadunek, dojazd do celu, rozładunek, powrót na pozycję domową. Nietypowe: robot wracający do bazy jest wolny, więc może od razu dostać nowe zlecenie bez dojazdu do domu. Z TO_PICKUP jest też ścieżka rezygnacji, czyli unassign z poprzedniego diagramu. Trzy stany jazdy korzystają ze wspólnej podmaszyny Ruch.

Slajd 6 – SM-3 Podmaszyna Ruch (Mikołaj, 35 s)

Podmaszyna Ruch. Planowanie to space-time A-star z tablicą rezerwacji; przy wyjściu robot rezerwuje trasę. W Jeździe przesuwa się o jedno pole na tick. Gdy trasy nie ma, przechodzi do Oczekiwania, planFailures rośnie i po ticku planuje ponownie. Jeśli plan stał się nieaktualny, bo inne roboty zmieniły rezerwacje, robot zwalnia swoje rezerwacje i planuje od nowa. Robot blokujący korytarz po yieldAfter tickach zjeżdża na sąsiednie wolne pole. Dwa wyjścia: u celu albo rezygnacja po N nieudanych planach.

Slajd 7 – SM-4 Magazyn (Mikołaj, 55 s)

Magazyn. Tu analiza stanów zmieniła model: pole archived typu Boolean z Checkpointu 1 nie wystarczało, bo klient nie powinien widzieć magazynu w trakcie projektowania. Mamy status DRAFT, ACTIVE, ARCHIVED. Szkic ma podstany Niepoprawny i Poprawny wyliczane przez validate po każdej zmianie; opublikować można tylko poprawny. Edycja opublikowanego magazynu jest bezpieczna, bo przebiegi pracują na zamrożonej migawce, a numer wersji chroni przed nadpisaniem cudzych zmian. Usunąć można tylko archiwalny magazyn bez przebiegów. Piąty obiekt, konto użytkownika, jest w dokumencie: po rejestracji aktywne z rolą Klient, administrator może zmienić rolę lub dezaktywować konto, z warunkiem „nie ostatni administrator”. Konta nie są usuwane, bo są właścicielami przebiegów, więc diagram nie ma stanu końcowego. Anna pokaże krok symulacji.

Slajd 8 – AD-1 Krok symulacji, część 1 (Anna, 35 s)

Krok symulacji to najczęściej wykonywana funkcjonalność: zegar wyzwala go co tickIntervalMs dla każdego przebiegu RUNNING; pauza zatrzymuje timer. Trzy tory: RunManager, silnik i publikacja. RunManager zajmuje blokadę przebiegu i wywołuje Step; pauza i stop czekają na tę blokadę, więc nie przerywają kroku w połowie. Silnik nie wykonuje żadnego wejścia-wyjścia. Najpierw zwiększa licznik ticków i zwalnia zlecenia, których releaseTick już minął, do kolejki FIFO. Potem w pętli przydziela pierwsze zlecenie z kolejki najbliższemu wolnemu robotowi według odległości BFS, dopóki są zlecenia i wolne roboty.

Slajd 9 – AD-1 Krok symulacji, część 2 (Anna, 30 s)

Druga część to planowanie tras: region iteracyjny dla każdego robota bez aktualnego planu. Trasa jest wyznaczana algorytmem space-time A-star względem tablicy rezerwacji. Trasa znaleziona, to rezerwacja; brak trasy, to robot czeka w miejscu i rośnie planFailures. Po N nieudanych próbach unassign zwraca zlecenie do kolejki, a robot wraca do bazy; to ścieżka nietypowa z diagramów stanów zlecenia i robota.

Slajd 10 – AD-1 Krok symulacji, część 3 (Anna, 35 s)

Trzecia część: każdy robot przesuwa się o jedno pole albo czeka, silnik obsługuje załadunek i rozładunek i aktualizuje statusy zleceń. SafetyMonitor sprawdza konflikty wierzchołkowe i krawędziowe; naruszenie daje ENGINE_ERROR, bo wynik byłby niewiarygodny. Potem metryki i StallDetector oraz warunek końca: wszystkie zlecenia zakończone, limit ticków albo zastój. Silnik zwraca TickUpdate. Jeśli przebieg trwa, LiveUpdatePublisher wysyła go przez SignalR do grupy przebiegu; jeśli się skończył, RunManager zatrzymuje timer, ResultService zapisuje wynik i wysyłany jest run_status. Roboty są obsługiwane w stałej kolejności według identyfikatora, więc te same dane zawsze dają ten sam wynik. Patrick pokaże start przebiegu.

Slajd 11 – Trzy bramki, migawka, rozwidlenie (Patrick, 50 s)

Uruchomienie przebiegu. Klient wysyła POST start z tokenem JWT. Trzy bramki po kolei: brak uprawnień to 403, status inny niż DRAFT to 409, na przykład podwójne kliknięcie, błędy walidacji to 422 z listą. Ważne, że WarehouseService zamraża migawkę magazynu przed walidacją, więc walidacja i cały przebieg używają tego samego układu. Po sukcesie status RUNNING, RunManager tworzy silnik i tu jest rozwidlenie: pętla ticków rusza w tle, a klient dostaje 202, łączy się z hubem i dostaje migawkę stanu do narysowania. Pozostałe aktywności są w dokumencie: dodanie zlecenia z trzema walidacjami i pętlą formularza, zapis magazynu z kontrolą wersji i równoległą walidacją trzech grup reguł oraz podgląd na żywo z regionem przerywalnym i ponownym łączeniem.

Slajd 12 – Wnioski z analizy stanów (Patrick, 35 s)

Analiza stanów miała poprawić strukturę metod i tak się stało. Magazyn dostał status zamiast flagi archived oraz operacje publish i restore. Zlecenie dostało unassign. Użytkownik stan Dezaktywowane i reactivate. Przebieg dostał INTERRUPTED i metodę oznaczającą przerwane przebiegi przy starcie serwera. Robot ma wspólną podmaszynę ruchu z licznikiem nieudanych planów. Zaktualizowane diagramy klas i historyjki pokażemy na Checkpoincie 4, zgodnie z zadaniem. Dziękujemy, czy są pytania?